

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

**Тема: РАЗРАБОТКА ВИЗУАЛИЗАТОРА
ТОПОЛОГИЧЕСКОЙ СОРТИРОВКИ НА ЯЗЫКЕ JAVA**

Студент гр. 4382

Д.А. Баневич

Студент гр. 4382

М.О. Боков

Студент гр. 4383

Я.В. Овчаренко

Руководитель

Р.П. Шестопапов

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Баневич Дмитрий Алексеевич Группа: 4382

Студент: Боков Максим Олегович Группа: 4382

Студент: Овчаренко Ярослав Владимирович Группа: 4383

Тема практики: Разработка визуализатора топологической сортировки на языке Java

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на Java с графическим интерфейсом.

Алгоритм: топологическая сортировка ориентированного графа методом Кана.

Сроки прохождения практики: 06.07.2026 - 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	Д.А. Баневич
---------	--------------

Студент	М.О. Боков
---------	------------

Студент	Я.В. Овчаренко
---------	----------------

Руководитель	Р.П. Шестопапов
--------------	-----------------

АННОТАЦИЯ

В отчёте представлены результаты учебной практики по направлению «Новые языки программирования». Описаны выполнение индивидуального задания на Java, подготовка спецификации и командная разработка визуализатора топологической сортировки. Приложение создано на Java Swing и поддерживает редактирование и загрузку графа, пошаговое и автоматическое выполнение алгоритма Кана, переход назад, обнаружение цикла, сохранение результатов и обработку некорректных файлов. Приведены архитектура, основные методы и результаты тестирования.

SUMMARY

The report presents the results of educational practice in new programming languages. It describes an individual Java assignment, software specification, and iterative team development of a topological sorting visualizer. The Java Swing application supports graph editing and file loading, step-by-step and automatic execution of Kahn's algorithm, backward navigation, cycle detection, saving, and input validation. The architecture, main methods, and testing results are also presented.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	7
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1 Вводное задание: перенос структуры данных Rore на Java.....	8
2.2 Составление спецификации программы и плана разработки.....	9
2.3 Командная итеративная разработка приложения	13
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	22
4 ОТЗЫВ РУКОВОДИТЕЛЯ	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25
ПРИЛОЖЕНИЕ А. Форматы входных и выходных данных.....	26
ПРИЛОЖЕНИЕ Б. Псевдокод алгоритма Кана	28

ВВЕДЕНИЕ

Современная разработка программного обеспечения требует не только знания алгоритмов и структур данных, но и умения переносить алгоритмическую идею в законченное пользовательское приложение. Учебная практика по направлению «Новые языки программирования» была посвящена освоению языка Java и применению его средств для командной разработки визуализатора алгоритма. В качестве предметной области была выбрана визуализация работы с ориентированными графами и построение топологического порядка вершин.

Топологическая сортировка используется в задачах, где между объектами заданы отношения предшествования. К таким задачам относятся определение порядка сборки программных модулей, планирование работ с зависимостями, составление учебных планов, разрешение зависимостей пакетов и обработка ориентированных ациклических графов. Графическое представление алгоритма позволяет наблюдать не только итоговый порядок, но и причины, по которым очередная вершина может быть выбрана на конкретном шаге.

Целью практики являлось освоение основных средств языка Java и разработка законченного графического приложения, демонстрирующего алгоритм топологической сортировки. Для достижения цели требовалось выполнить индивидуальное вводное задание, подготовить и согласовать спецификацию мини-проекта, распределить обязанности между участниками бригады, реализовать приложение в нескольких последовательных версиях, организовать работу в общем репозитории и провести тестирование полученного результата [1].

Разработка велась итеративно: прототип интерфейса, альфа-версия с загрузкой графа, бета-версия с пошаговой визуализацией и финальный релиз. Такой порядок позволил регулярно демонстрировать результат и своевременно уточнять требования.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Ориентированный граф состоит из множества вершин и множества рёбер, каждое из которых имеет заданное направление. Ребро $u \rightarrow v$ задаёт отношение предшествования: вершина u должна быть обработана раньше вершины v . Если граф не содержит направленных циклов, для него можно построить топологический порядок. В таком порядке для любого ребра $u \rightarrow v$ вершина u расположена раньше вершины v .

Одним из распространённых способов построения топологического порядка является алгоритм Кана. Для каждой вершины сначала вычисляется входящая степень - число рёбер, заканчивающихся в этой вершине. Все вершины с нулевой входящей степенью помещаются в очередь. Затем вершина извлекается из очереди, добавляется в результат, а входящие степени её соседей уменьшаются. Если степень соседа становится равной нулю, сосед добавляется в очередь. Если в итоговый список попали не все вершины, это означает наличие направленного цикла [2].

Визуализация алгоритма должна показывать состояние графа и служебных структур после каждого логического действия. Пользователю необходимо видеть текущую очередь, рабочие входящие степени, уже сформированную часть результата, выбранную вершину и обрабатываемое ребро. Такое представление делает алгоритм понятнее и позволяет использовать программу как учебное средство при изучении графовых алгоритмов.

Приложение ориентировано на графы умеренного размера, которые удобно анализировать визуально. Для графов, выходящих за размеры рабочей области, требуется прокрутка. Все вершины должны иметь целочисленные идентификаторы, направления рёбер должны быть подписаны, а некорректные операции - добавление петли, повторного ребра или ребра к отсутствующей вершине - должны отклоняться.

1.2 Задачи практики

Для достижения поставленной цели были определены следующие задачи:

1. Изучить основные конструкции Java, объектно-ориентированный подход, коллекции, обработку исключений и средства ввода-вывода;
2. Выполнить индивидуальное вводное задание и подготовить исполнимый JAR-файл;
3. Составить спецификацию визуализатора топологической сортировки, определить формат данных и ограничения;
4. Разработать план итеративной реализации с отдельными этапами прототипа, альфа-, бета- и финальной версии;
5. Организовать совместную работу бригады в репозитории и распределить зоны ответственности;
6. Реализовать модель ориентированного графа и алгоритм Кана с обнаружением циклов;
7. Создать графический интерфейс на Swing и пользовательские операции редактирования графа;
- ё8. Реализовать пошаговую визуализацию, возврат к предыдущему шагу и автоматическое выполнение с интервалом;
9. Реализовать загрузку и сохранение графа, сохранение результата и обработку ошибок файлов;
10. Провести функциональное тестирование алгоритма, интерфейса и исключительных ситуаций;
11. Собрать приложение в исполнимый JAR-файл и подготовить отчётную документацию.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1 Вводное задание: перенос структуры данных Rope на Java

В качестве индивидуального вводного задания была выбрана часть лабораторной работы по структуре данных Rope. Rope представляет строку в виде бинарного дерева. Листовые узлы содержат фрагменты строки, а внутренние узлы содержат ссылки на левое и правое поддерево и вес левого поддерева. Вес используется для выбора направления перехода при поиске символа по индексу.

В Java были реализованы классы RopeNode и Rope. Узлы создаются как отдельные объекты в динамической памяти и связываются ссылками left и right; библиотечные контейнеры для хранения структуры дерева не используются. Исходная строка рекурсивно делится на части, поэтому формируется сбалансированное дерево. Максимальный размер фрагмента строки в листе установлен равным восьми символам.

Метод index получает целочисленный индекс. Во внутреннем узле индекс сравнивается с weight. Если индекс меньше веса, поиск продолжается в левом поддереве. В противном случае из индекса вычитается weight, после чего выполняется переход вправо. После достижения листа возвращается код найденного символа. Для отрицательного индекса или индекса за пределами строки возвращается значение -1. Программа была собрана в исполнимый файл RopeIndex.jar.

Пример запуска программы:

Введите исходную строку:

topological sorting

Структура Rope:

[внутренние узлы и листовые фрагменты строки]

Введите индекс символа:

4

Символ: l

Результат показывает, что поиск выполняется через структуру дерева, а не прямым обращением к исходному массиву символов. Вводное задание позволило закрепить создание связанных объектов, рекурсию, инкапсуляцию, обработку ошибочного ввода и сборку приложения в JAR-файл.

2.2 Составление спецификации программы и плана разработки

До начала реализации была подготовлена спецификация приложения «Визуализатор топологической сортировки». Документ определил назначение программы, набор пользовательских функций, ограничения, структуру интерфейса, форматы входных и выходных файлов, алгоритм визуализации, обработку ошибок, требования к репозиторию, план итераций, распределение ролей и план тестирования. Спецификация использовалась как основа для последовательного развития проекта и фиксации уточнений после промежуточных демонстраций [3].

2.2.1 Исходные требования к программе

2.2.1.1 Требования к вводу и редактированию графа

Граф должен создаваться вручную либо загружаться из текстового файла. При ручном вводе вершина добавляется щелчком мыши по рабочей области, а ребро создаётся последовательным выбором начальной и конечной вершин. В финальной версии дополнительно реализованы удаление вершины вместе со всеми инцидентными рёбрами, удаление отдельного ребра и перемещение отдельной вершины мышью с автоматической перерисовкой связанных рёбер.

Файл графа содержит секцию VERTICES с количеством вершин и строками вида <id> <x> <y>, после которой следует секция EDGES со строками вида <id начала> <id конца>. Пустые строки и строки, начинающиеся с символа #, игнорируются. Идентификаторы и координаты должны быть целыми неотрицательными числами.

2.2.1.2 Требования к визуализации

Вершины отображаются кругами с идентификаторами, рёбра - направленными стрелками с подписями from -> to. Рядом с вершиной

выводится её текущая входящая степень. Во время выполнения алгоритма выбранная вершина, текущее ребро, вершины в очереди и уже обработанные вершины выделяются различными цветами и толщиной контура. Если граф выходит за границы видимой области, используются горизонтальная и вертикальная полосы прокрутки.

Справа от холста располагается панель управления алгоритмом, содержащая номер шага, текущий элемент, очередь, текущий результат и таблицу входящих степеней. В нижней части окна размещается подробное текстовое пояснение текущего действия. После замечаний к альфа-версии область пояснений была увеличена и помещена в JScrollPane, чтобы длинное сообщение всегда можно было прочитать полностью.

2.2.1.3 Требования к управлению алгоритмом

Алгоритм должен запускаться для текущей модели графа. В альфа-версии требовалось вывести только конечный результат. В бета-версии были добавлены логические шаги, кнопка перехода вперёд, отображение очереди и рабочих степеней, а также сохранение результата. Финальный релиз должен позволять переходить к предыдущему шагу, запускать последовательность автоматически с задаваемым интервалом и останавливать выполнение.

Для графа с циклом программа не должна формировать ложный топологический порядок. Наличие цикла определяется по числу обработанных вершин: если оно меньше общего числа вершин, алгоритм завершает визуализацию сообщением о невозможности сортировки.

2.2.1.4 Требования к сохранению и обработке ошибок

Текущий граф должен сохраняться в том же текстовом формате, который используется для загрузки. После работы алгоритма пользователь может сохранить отдельный файл с числом вершин и рёбер, количеством шагов, начальными входящими степенями и итоговым порядком либо сообщением о цикле.

Приложение не должно аварийно завершаться при пустом файле, неправильных заголовках секций, неверном количестве полей, нечисловых

значениях, отрицательных идентификаторах, повторяющихся вершинах и рёбрах, петлях, ссылках на несуществующие вершины, лишних данных, слишком большом файле и ошибках доступа. Пользователь должен получать понятное сообщение с номером строки, если ошибка относится к содержимому файла. Эскиз интерфейса приведён на рис. 1.

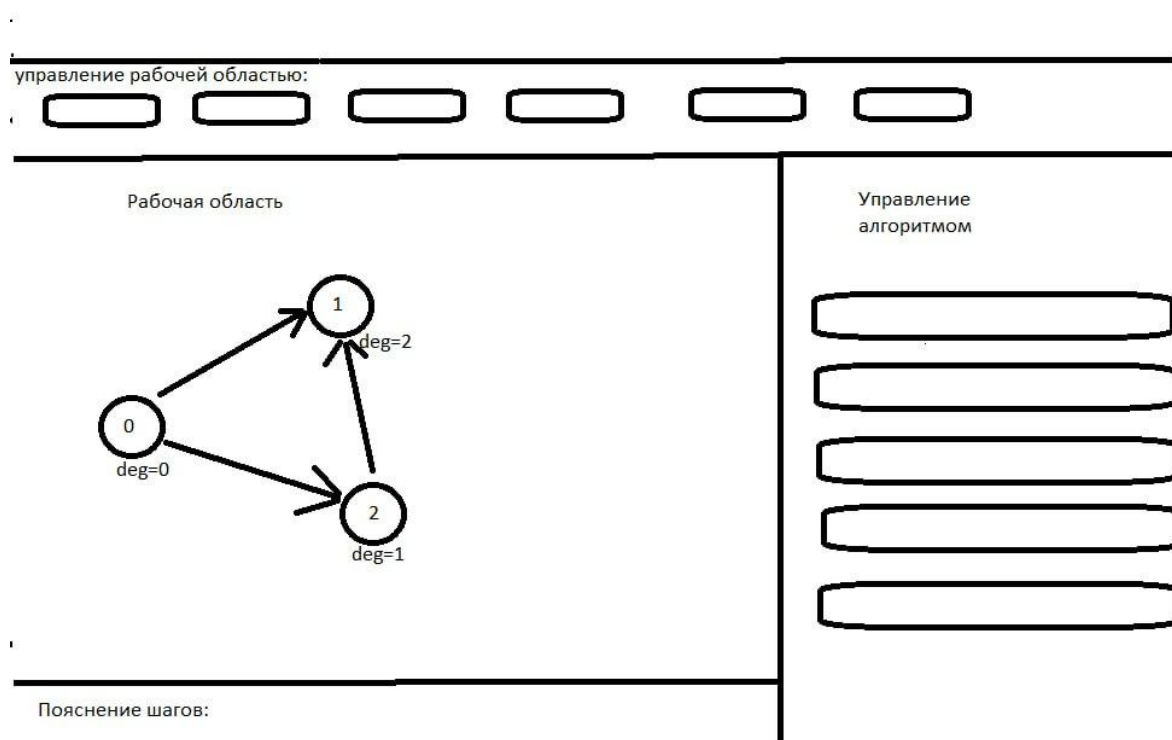


Рисунок 1 - Эскиз пользовательского интерфейса, подготовленный на этапе спецификации

2.2.2 План разработки и распределение ролей в бригаде

Разработка была разделена на версии так, чтобы на каждой демонстрации можно было показать законченный прирост функциональности. Первоначальный план и фактически реализованные результаты приведены в таблице 1.

Таблица 1 - План итеративной разработки

Этап	Срок	Планируемый результат	Фактический результат
Спецификация и прототип	до 10.07.2026	Структура окна, холст, панели, демонстрационный граф, кнопки-заглушки	Подготовлены спецификация, репозиторий и работоспособный прототип интерфейса
Альфа-версия	до 13.07.2026	Загрузка графа, отрисовка, запуск алгоритма, конечный порядок, базовая проверка цикла	Реализованы загрузка и сохранение графа, алгоритм Кана, результат и обработка цикла
Бета-версия	до 15.07.2026	Пошаговое выполнение, подсветка, очередь, степени, сохранение результата	Реализованы снимки состояний, переход по шагам и визуальное отображение состояния
Финальный релиз	до 17.07.2026	Шаг назад, автозапуск, остановка, ошибки, JAR и отчёт	Добавлены все функции релиза, редактирование графа, расширенная валидация и Release 1.0

Ответственность была распределена в соответствии с архитектурными частями приложения. Такое распределение позволило каждому участнику объяснять собственную часть и фиксировать вклад отдельными коммитами. Распределение ролей приведено в таблице 2.

Таблица 2 - Распределение ролей в бригаде

Участник	Группа	Зона ответственности
Баневич Дмитрий	4382	Модель графа, классы Vertex и DirectedEdge, GraphModel, алгоритм Кана, обнаружение цикла, AlgorithmStep и подготовка шагов визуализации
Боков Максим	4382	Главное окно, GraphPanel, элементы Swing, отрисовка вершин и стрелок, обработка мыши, подсветка и управление состояниями интерфейса
Овчаренко Ярослав	4383	GraphFileService, ResultFileService, загрузка и сохранение, обработка файловых ошибок, тестовые графы, README и документация

2.2.3 Уточнение требований после сдачи прототипа

После демонстрации прототипа было подтверждено, что кнопки на данном этапе могут оставаться частично неработающими, однако расположение основных областей должно соответствовать будущему приложению. Было принято решение сохранить отдельный холст с прокруткой, правую панель управления и нижнюю область пояснений. Также была зафиксирована необходимость подписывать направления рёбер и отображать входящие степени вершин.

2.2.4 Уточнение требований после сдачи альфа-версии

После альфа-версии были уточнены подписи элементов интерфейса. Кнопка «Открыть» была переименована в «Открыть граф», а «Очистить» - в «Очистить граф». Режим перемещения получил подпись «Переместить вершину».

Нижняя область пояснений была увеличена и снабжена полосами прокрутки, поскольку длинные сообщения не всегда помещались в видимую часть окна.

2.2.5 Уточнение требований после сдачи бета-версии

Для финальной версии было решено дополнительно реализовать удаление вершин и рёбер, хотя эти операции не являлись обязательной частью исходного задания. Также была усилена проверка файлов: введены ограничения на размер файла, число вершин и рёбер, диапазон координат и отсутствие лишних данных после секции рёбер. Переход назад реализован через хранение неизменяемых снимков состояния, а автоматический режим - с помощью Swing Timer.

2.3 Командная итеративная разработка приложения

Разработка выполнялась в общем репозитории. Изменения группировались по функциональным веткам и содержательным коммитам: модель графа, прототип интерфейса, алгоритм Кана, работа с файлами, интеграция альфа-версии, пошаговая визуализация и финальный релиз. Перед началом новой части использовалась актуальная версия основной ветки, после чего изменения объединялись через запрос на слияние. Такой порядок снизил риск перезаписи файлов и сделал историю проекта соответствующей реальным этапам разработки [4], [5].

Архитектура приложения разделена на модель данных, алгоритмический слой, файловые сервисы и графический интерфейс. MainFrame координирует действия пользователя, GraphPanel отвечает за отрисовку и обработку мыши, GraphModel хранит граф, TopologicalSorter формирует результат и последовательность шагов, а сервисные классы выполняют чтение и запись. Разделение позволило тестировать алгоритм независимо от интерфейса.

2.3.1 Структуры данных

Класс Vertex. Описывает одну вершину и хранит неизменяемый целочисленный идентификатор, а также изменяемые координаты x и y .

Координаты используются GraphPanel для отрисовки и перемещения объектов. Идентификатор остаётся постоянным после создания.

Класс DirectedEdge. Описывает ориентированное ребро через поля from и to. Оба значения являются идентификаторами вершин. Метод connects используется для поиска конкретного ребра и отклонения повторного добавления.

Класс GraphModel. Вершины хранятся в LinkedHashMap<Integer, Vertex>, где ключом является идентификатор, а значением - объект вершины. Это обеспечивает быстрый доступ по id и сохраняет порядок добавления. Рёбра хранятся в ArrayList<DirectedEdge>, что упрощает отрисовку и последовательный обход. Модель поддерживает добавление, удаление, очистку, замену графа, подсчёт входящей степени и выбор исходящих рёбер.

Класс AlgorithmStep. Хранит один снимок состояния алгоритма: описание действия, копию таблицы входящих степеней, очередь, текущий результат, выбранную вершину, текущее ребро и признаки завершения и успешности. Коллекции копируются и оборачиваются в неизменяемые представления. Поэтому переход назад не требует обратного вычисления: интерфейс просто отображает предыдущий снимок.

Класс TopologicalSortResult. Объединяет признак успешности, итоговый порядок, исходные входящие степени, текстовое сообщение и список шагов. Объект является связующим звеном между алгоритмическим слоем, интерфейсом и сервисом сохранения результата.

Для очереди алгоритма используется ArrayDeque<Integer>. По сравнению со списком данная структура предоставляет эффективные операции добавления в конец и извлечения из начала. Для текущего порядка и последовательности шагов используются ArrayList, поскольку элементы добавляются последовательно, а интерфейсу требуется доступ к шагу по индексу.

2.3.2 Основные методы

2.3.2.1 Реализация алгоритма Кана

Основной метод `TopologicalSorter.sort` получает `GraphModel` и формирует `TopologicalSortResult`. Сначала создаётся `LinkedHashMap` входящих степеней. Для каждой вершины записывается ноль, затем выполняется проход по всем рёбрам и увеличивается степень конечной вершины. Исходная таблица копируется, поскольку рабочие значения будут изменяться во время алгоритма.

Все вершины с нулевой степенью помещаются в `ArrayDeque`. После этого создаётся первый `AlgorithmStep`, который отражает рассчитанные степени и начальную очередь. В главном цикле вершина извлекается из очереди и добавляется в результат. Для каждого исходящего ребра уменьшается степень соседней вершины. Если новое значение равно нулю, сосед добавляется в очередь. После каждого такого действия создаётся отдельный снимок.

После опустошения очереди размер результата сравнивается с количеством вершин. Равенство означает успешную сортировку. Если размеры отличаются, в графе присутствует цикл и часть вершин не могла получить нулевую степень. В обоих случаях добавляется финальный шаг с пояснением.

Теоретическая сложность алгоритма Кана при хранении списков смежности составляет $O(V + E)$. В текущем учебном проекте исходящие рёбра выбираются фильтрацией общего списка рёбер, поэтому строгая верхняя оценка реализации может достигать $O(VE)$. Такое решение выбрано ради простоты модели и отрисовки и является приемлемым для предусмотренных спецификацией графов умеренного размера. Возможным развитием является дополнительное хранение списка смежности для каждой вершины.

2.3.2.2 Пошаговое и автоматическое выполнение

`MainFrame` хранит последний `TopologicalSortResult` и индекс текущего шага. Метод `displayCurrentStep` передаёт состояние в `GraphPanel`, обновляет

номер шага, подпись текущего элемента, очередь, результат, степени и текстовое пояснение. Кнопка «Следующий шаг» увеличивает индекс, а «Предыдущий шаг» уменьшает его. Благодаря сохранённым снимкам оба перехода выполняются без изменения исходной модели графа.

Автоматический режим реализован классом `javax.swing.Timer`. Интервал задаётся `JSpinner` в диапазоне от 200 до 10000 мс. По каждому событию таймера индекс шага увеличивается и состояние перерисовывается. Кнопка «Стоп» останавливает таймер, а элементы управления блокируются или разблокируются в зависимости от текущего режима.

2.3.2.3 Отрисовка и взаимодействие с графом

`GraphPanel` наследуется от `JPanel` и переопределяет `paintComponent`. Сначала рисуется фоновая сетка, затем стрелки и вершины. Для ребра вычисляются точки начала и конца на границах окружностей, после чего строится линия и треугольный наконечник. Подпись `from -> to` размещается около середины ребра.

Состояние визуализации определяет цвет и толщину контура. Текущая вершина и текущее ребро выделяются наиболее контрастно, вершины очереди и обработанные вершины получают отдельное оформление. Рядом с каждой вершиной выводится рабочее значение `deg`. Метод `updateCanvasSize` изменяет предпочтительный размер панели с учётом крайних координат, благодаря чему `JScrollPane` отображает необходимые полосы прокрутки.

Обработчик мыши поддерживает разные режимы: добавление вершины, последовательный выбор концов ребра, перемещение вершины, удаление вершины и удаление ребра. При перемещении изменяются координаты только выбранной вершины, а все связанные с ней рёбра автоматически перерисовываются. Для выбора ребра вычисляется расстояние от курсора до отрезка. После любого изменения графа результат прежнего запуска сбрасывается, поскольку старые шаги больше не соответствуют модели.

2.3.2.4 Работа с файлами

GraphFileService.load выполняет предварительную проверку пути, существования, типа, доступности и размера файла. Далее файл читается в UTF-8. Заголовки секций, количество строк и число полей проверяются последовательно. Ошибки формируются с номером строки. Установлены ограничения: не более 2 МБ, 2000 вершин, 10000 рёбер и координаты от 0 до 100000.

Метод save записывает вершины и рёбра в исходный формат, сохраняя идентификаторы и координаты. ResultFileService создаёт отдельный отчёт о работе алгоритма: количество элементов графа, число шагов, исходные степени и итоговый порядок либо причину невозможности сортировки. Операции записи выполняются через try-with-resources и BufferedWriter.

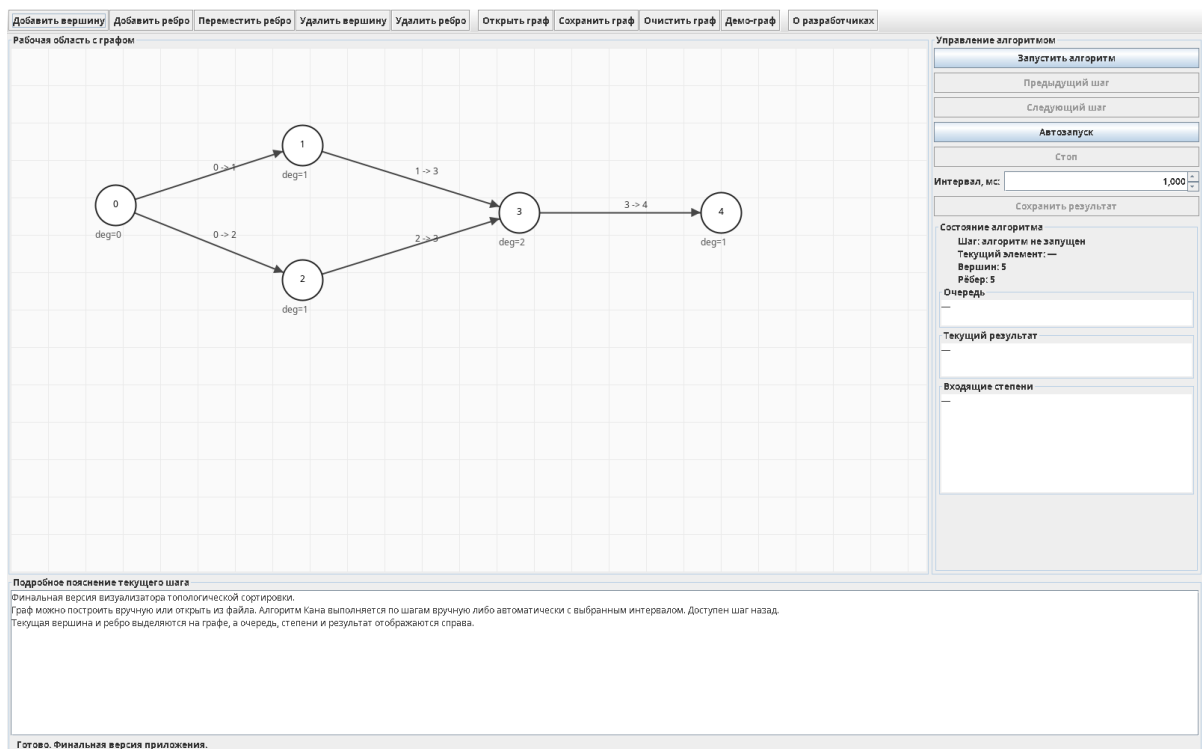


Рисунок 2 - Главное окно финальной версии до запуска алгоритма

На рис. 2 показано главное окно Release 1.0. В верхней панели доступны операции редактирования, загрузки и сохранения. Справа расположены элементы управления и области состояния, снизу - прокручиваемое пояснение.

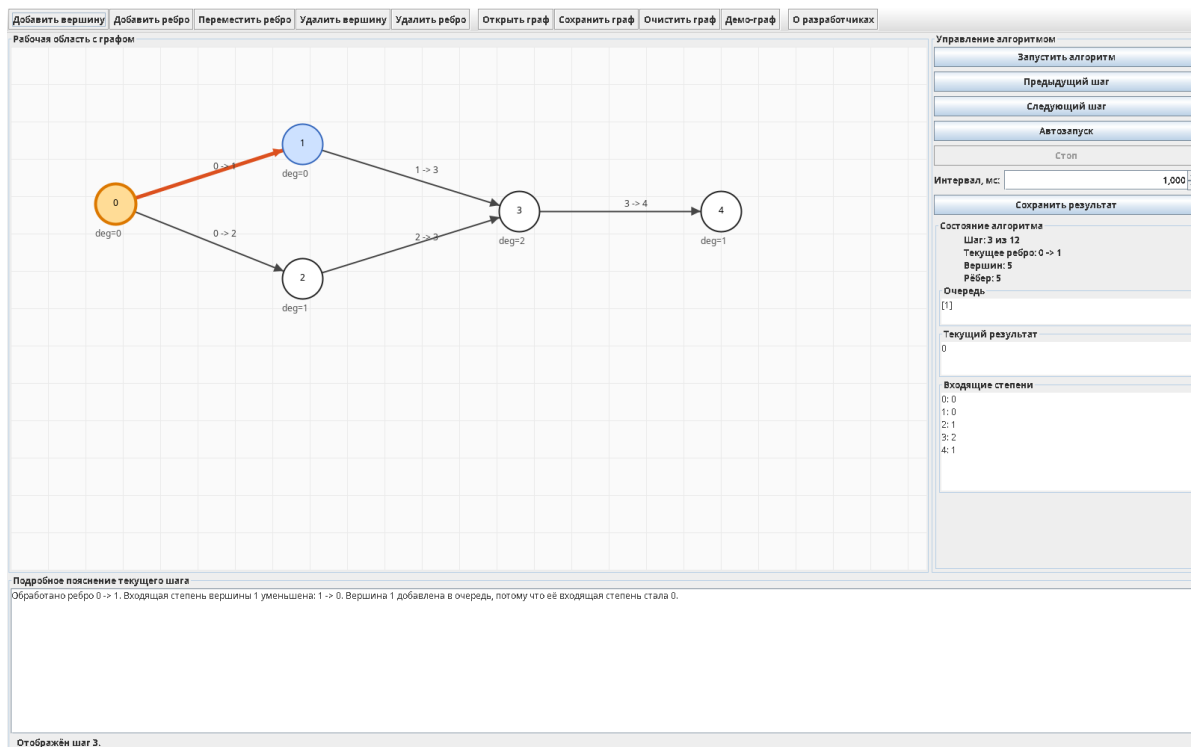


Рисунок 3 - Отображение шага обработки ребра 0 -> 1

На рис. 3 текущая вершина 0 и ребро 0 -> 1 выделены цветом. В правой панели показаны очередь [1], текущий результат 0 и изменившиеся входящие степени. Нижняя область содержит текстовую причину изменения состояния.

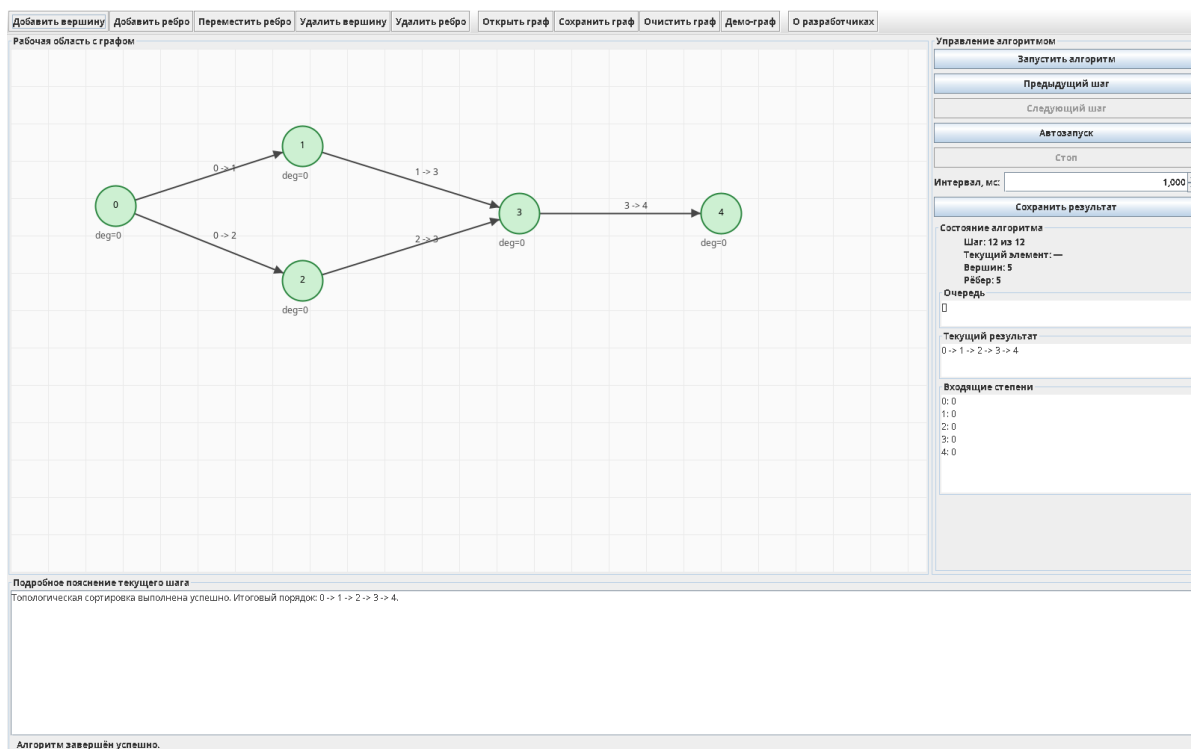


Рисунок 4 - Успешное завершение топологической сортировки

После обработки всех вершин программа отображает итоговый порядок и нулевые рабочие входящие степени. Обработанные вершины выделяются зелёным цветом, а статус сообщает об успешном завершении (рис. 4).

2.3.3 Тестирование

2.3.3.1 План тестирования

Тестирование было разделено на проверку алгоритма, графического интерфейса, файловых операций и исключительных ситуаций. Для повторяемых проверок подготовлены файлы graph_example.txt, graph_cycle.txt, graph_disconnected.txt, graph_invalid_header.txt, graph_invalid_edge.txt и graph_extra_data.txt. Основные сценарии приведены в таблице 3.

Таблица 3 - План и результаты функционального тестирования

№	Сценарий	Ожидаемый результат	Результат
1	Простой DAG	Получен корректный порядок всех пяти вершин	Пройден: 0 -> 1 -> 2 -> 3 -> 4
2	Несколько начальных вершин	Все вершины с deg=0 находятся в очереди	Пройден
3	Несвязный DAG	В порядок входят вершины всех компонент	Пройден: 0 -> 2 -> 4 -> 1 -> 3 -> 5
4	Цикл 0 -> 1 -> 2 -> 0	Вывод сообщения о невозможности сортировки	Пройден
5	Пустой граф	Предупреждение без аварийного завершения	Пройден
6	Неверный заголовок файла	Файл отклонён с номером строки	Пройден
7	Ребро к отсутствующей вершине	Файл отклонён	Пройден
8	Лишние данные после EDGES	Файл отклонён	Пройден
9	Переход вперёд и назад	Полностью восстанавливается состояние шага	Пройден
10	Автозапуск и остановка	Шаги меняются с интервалом, «Стоп» останавливает таймер	Пройден вручную
11	Удаление вершины и ребра	Удаляются выбранные элементы и связанные рёбра	Пройден
12	Сохранение и повторная загрузка	Число вершин и рёбер сохраняется	Пройден
13	Граф за пределами окна	Появляются горизонтальная и вертикальная прокрутки	Пройден вручную

2.3.3.2 Тестирование графического интерфейса

Интерфейс проверялся при начальном состоянии, в процессе алгоритма и после завершения. Было подтверждено, что кнопки перехода назад и вперёд активируются только при наличии соответствующего шага, автозапуск блокирует несовместимые элементы управления, а кнопка «Стоп» доступна только во время работы таймера. Изменение графа сбрасывает устаревшую визуализацию.

Проверена читаемость подписей вершин и направлений рёбер. Нижнее пояснение переносит длинный текст по словам и имеет вертикальную и горизонтальную прокрутку. Холст находится в JScrollPane, поэтому при

координатах за пределами видимой области граф остаётся доступным. Состояния на рис. 2-4 подтверждают корректное обновление интерфейса.

2.3.3.3 Тестирование алгоритма

Для простого ориентированного ациклического графа было сформировано 12 шагов визуализации и итоговый порядок $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. Корректность проверена для каждого ребра: его начальная вершина находится раньше конечной. Для несвязного DAG в результат вошли все шесть вершин. Наличие нескольких допустимых топологических порядков не считается ошибкой.

Для графа $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 0$ начальная очередь оказалась пустой, поскольку входящая степень каждой вершины равна единице. Размер результата остался меньше числа вершин, поэтому был создан финальный шаг с сообщением о цикле. Это состояние показано на рис. 5.

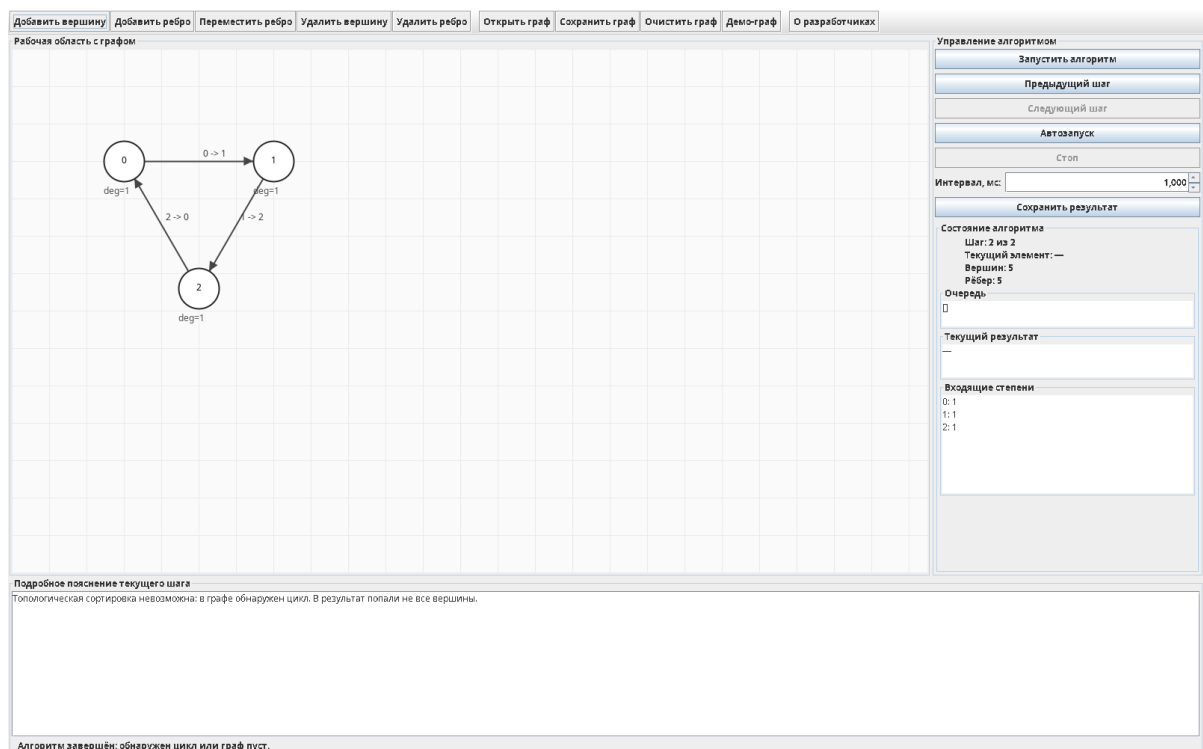


Рисунок 5 - Обнаружение направленного цикла

2.3.3.4 Тестирование файлов и исключительных ситуаций

Автоматическая проверка подтвердила успешное чтение корректных файлов, сохранение и повторную загрузку без изменения числа вершин и рёбер. Файлы с неверным заголовком, ссылкой на несуществующую вершину

и лишними данными были отклонены. Исключения перехватываются MainFrame и отображаются через JOptionPane, поэтому окно приложения остаётся открытым и пользователь может выбрать другой файл.

Отдельно проверены операции удаления. После удаления ребра уменьшается количество элементов списка edges. Удаление вершины удаляет все рёбра, для которых она является началом или концом. Это предотвращает появление ссылок на отсутствующие объекты и сохраняет целостность GraphModel.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Основным языком разработки выбран Java 17. Язык предоставляет статическую типизацию, объектно-ориентированную модель, автоматическое управление памятью, стандартные коллекции, средства обработки исключений и встроенные классы для работы с файлами. Совместимость с Java 17 задаётся при сборке параметром `--release 17` [6]. Используемые технологии и их назначение приведены в таблице 4.

Таблица 4 - Используемые технологии и средства

Технология	Назначение в проекте
Java 17	Модель графа, алгоритм, сервисы ввода-вывода, обработка исключений и точка входа
Swing	Главное окно, панели, кнопки, поля состояния, выбор файлов, диалоги, таймер и прокрутка
Java2D	Отрисовка сетки, вершин, подписей, линий, стрелок и цветовых состояний
Java Collections Framework	LinkedHashMap, ArrayList, ArrayDeque, Set и неизменяемые копии состояний
java.nio.file	Чтение и запись UTF-8 файлов, проверка существования, размера и прав доступа
Git и GitHub	Совместная разработка, история изменений, ветки, коммиты и объединение частей
JAR	Доставка исполнимой версии, запускаемой вне среды разработки
Stepik и официальная документация Java	Самостоятельное изучение синтаксиса и стандартных библиотек

Для построения интерфейса использованы JFrame, JPanel, JToolBar, JButton, JTextArea, JLabel, JSpinner, JScrollPane, JFileChooser и JOptionPane. Автоматическое выполнение реализовано Swing Timer, который генерирует события в потоке обработки интерфейса. Это исключает прямое обновление компонентов из фонового потока [7].

Для сборки подготовлены сценарии build.bat и build.sh. Сначала исходные файлы компилируются в каталог out, затем команда `jar cfe` создаёт архив с указанием главного класса `ru.leti.toposort.Main`. Полученный файл `toposort-visualizer-release.jar` запускается командой `java -jar` и не требует среды разработки.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на

_____.

Раздел заполняется после защиты финальной версии и отчёта. Для кафедральной темы достаточно указанной формулировки с выставленной оценкой.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики были освоены основные средства языка Java и выполнен полный цикл разработки учебного программного продукта. Индивидуальное задание позволило закрепить работу со связанными объектами и сборку JAR-файла. Для командного мини-проекта была подготовлена и согласована спецификация, определены форматы данных, требования к визуализации, план версий и зоны ответственности.

В результате создан визуализатор топологической сортировки ориентированного графа. Приложение позволяет создавать и редактировать граф вручную, загружать и сохранять его в текстовом формате, запускать алгоритм Кана, просматривать выполнение по шагам, возвращаться назад, использовать автоматический режим и сохранять итоговый результат. Текущая вершина и ребро выделяются, а очередь, степени и построенная часть порядка отображаются в отдельных областях интерфейса.

Была реализована обработка циклических графов и некорректных файлов. Программа проверяет структуру входных данных, количество элементов, ссылки между вершинами, петли, повторы и ограничения размера. При ошибке пользователь получает понятное сообщение, а приложение продолжает работу.

Разработка велась итеративно от прототипа до Release 1.0. Основная цель практики достигнута: алгоритмическая задача была преобразована в законченное графическое приложение, собранное в исполнимый JAR-файл и размещённое в общем репозитории. Возможными направлениями дальнейшего развития являются хранение списков смежности для улучшения асимптотики, изменение направления существующего ребра одной командой и автоматическая раскладка вершин большого графа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Java. Базовый курс [Электронный ресурс]. URL: <https://stepik.org/course/Java-Базовый-курс-187/syllabus> (дата обращения: 06.07.2026).
2. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. - 3-е изд. - М.: Вильямс, 2013.
3. Спецификация и план разработки приложения «Визуализатор топологической сортировки». Учебная практика «Новые языки программирования», 2026.
4. Git Documentation [Электронный ресурс]. URL: <https://git-scm.com/doc> (дата обращения: 10.07.2026).
5. GitHub Docs: Collaborating with pull requests [Электронный ресурс]. URL: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests> (дата обращения: 10.07.2026).
6. Java Platform, Standard Edition 17 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/> (дата обращения: 10.07.2026).
7. Creating a GUI With Swing [Электронный ресурс]. URL: <https://docs.oracle.com/javase/tutorial/uiswing/> (дата обращения: 10.07.2026).

ПРИЛОЖЕНИЕ А. ФОРМАТЫ ВХОДНЫХ И ВЫХОДНЫХ ДАННЫХ

Общий формат входного файла графа:

```
VERTICES <количество вершин>
<id вершины> <x> <y>
...
EDGES <количество рёбер>
<id начала> <id конца>
...
```

Пример корректного входного файла:

```
VERTICES 5
0 100 100
1 300 100
2 100 260
3 320 260
4 520 180
EDGES 5
0 1
0 2
1 3
2 3
3 4
```

Пример выходного файла для успешной сортировки:

```
Визуализатор топологической сортировки - результат
Количество вершин: 5
Количество рёбер: 5
Количество шагов визуализации: 12
```

Начальные входящие степени:

```
0: 0
1: 1
2: 1
3: 2
4: 1
```

Топологическая сортировка выполнена успешно.

Топологический порядок:

0 -> 1 -> 2 -> 3 -> 4

Пример выходного файла при наличии цикла:

Топологическая сортировка невозможна.

Причина: Топологическая сортировка невозможна: в графе обнаружен цикл.

Частично построенный порядок: —

ПРИЛОЖЕНИЕ Б. ПСЕВДОКОД АЛГОРИТМА КАНА

для каждой вершины v :

$\text{indegree}[v] = 0$

для каждого ребра $u \rightarrow v$:

$\text{indegree}[v] = \text{indegree}[v] + 1$

$\text{queue} =$ пустая очередь

для каждой вершины v :

если $\text{indegree}[v] == 0$:

добавить v в queue

$\text{result} =$ пустой список

пока queue не пуста:

$\text{current} =$ извлечь вершину из queue

добавить current в result

для каждого ребра $\text{current} \rightarrow \text{neighbor}$:

$\text{indegree}[\text{neighbor}] = \text{indegree}[\text{neighbor}] - 1$

если $\text{indegree}[\text{neighbor}] == 0$:

добавить neighbor в queue

если размер result меньше количества вершин:

сообщить о наличии цикла

иначе:

вывести result

В приложении после инициализации, выбора вершины и обработки каждого ребра создаётся объект `AlgorithmStep`. Последовательность объектов используется интерфейсом для движения вперёд, назад и автоматического воспроизведения.